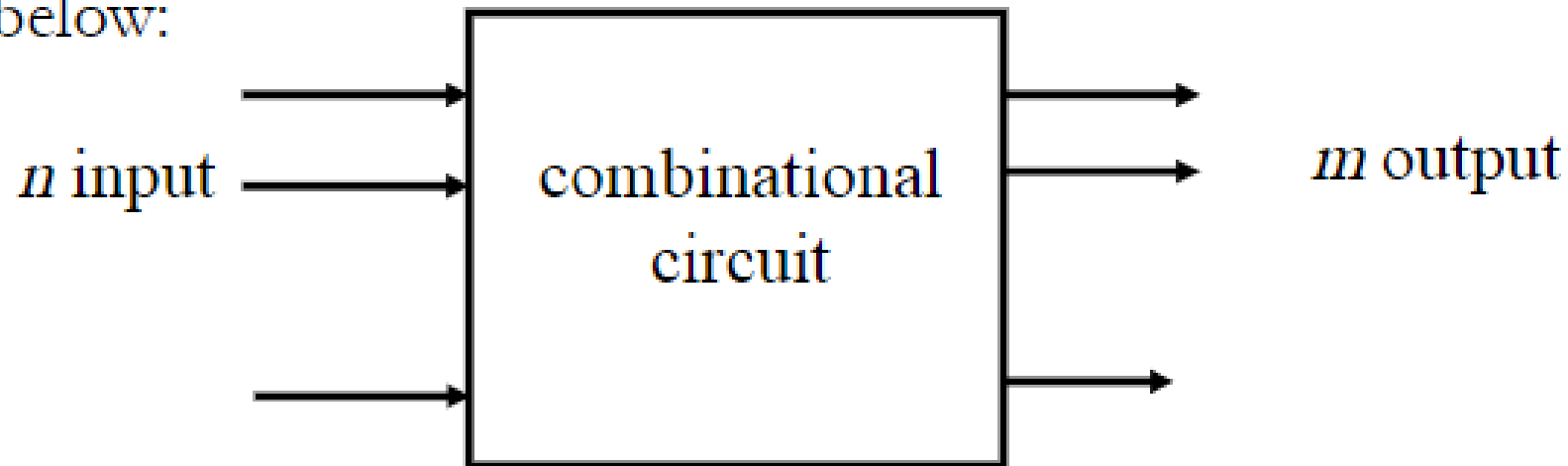


Combinational Circuit

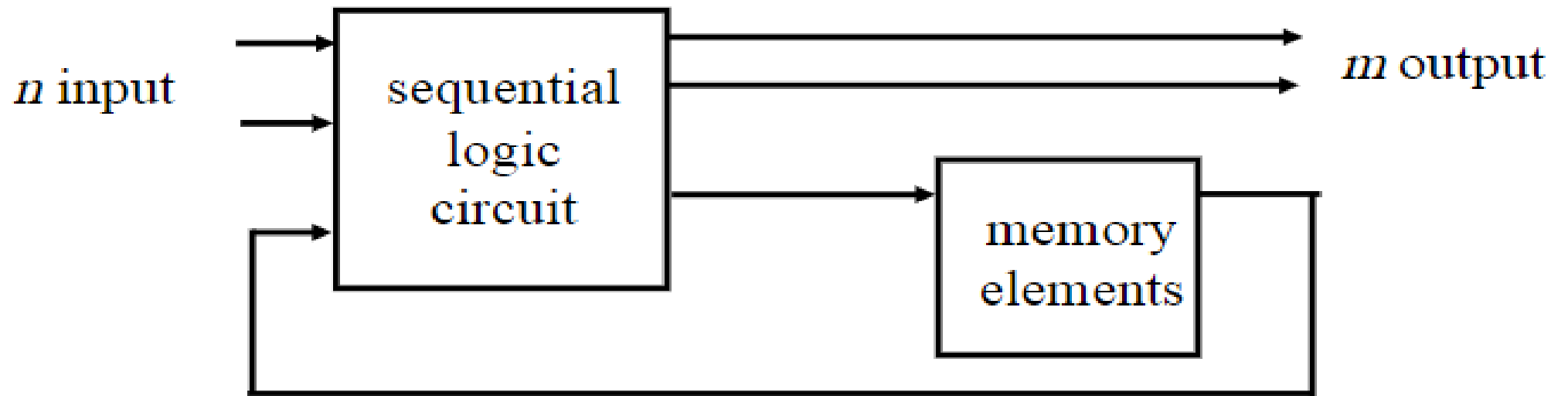
- Combinational Logic circuit contains logic gates where its output is determined by the combination of the current input, regardless of the output or the prior combination of input.
- Basically, combinational circuit can be depicted by Diagram 1 below:



- Examples of Combinational circuits in the computer system are decoder, parallel adder, and multiplexer.

Sequential Circuit

- Sequential Logic Circuit contains logic gates arranged in parallel and its output is not only determined by the combination of the current input, but also the prior output.
- The circuit also contains memory elements that enable it to store the information of the prior output.



- Examples of sequential circuits in the computer system are like registers, counters and serial adders.

Some Examples of Combinational Circuit:

Parallel Adder, Decoder, etc

- The circuits learnt in chapter 3 are combinational circuits. The steps to design combinational circuits are as the following:
 1. Understand the problem
 2. Determine the number of **input** and **output variables** that are needed
 3. Give symbols for the stated input and output
 4. Construct a **truth table** that defines the relationship between the input and output
 5. Obtain the **Boolean function** or the logical expression from the truth table in (4) using Karnaugh Map or other known methods.
 6. Draw a **logic circuit** based on the expression obtained from (5) above.



4

Addition/Subtraction Circuits

Remember

► Example:

$$\begin{array}{r}
 A = 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \\
 + \ B = 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \\
 \hline
 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 0
 \end{array}$$

- Each bit has two outputs: sum and carry.
- The carry out of current bit is the carry in to the next bit.

X	Y	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

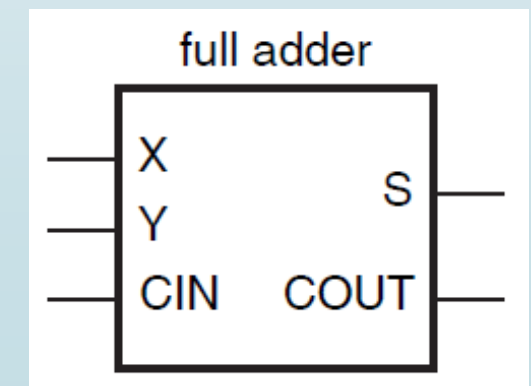
Full 1-bit Adder

► Example:

$$\begin{array}{r}
 A = 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \\
 + B = 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \\
 \hline
 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 0
 \end{array}$$

- The full 1-bit adder is a circuit that perform a **single bit to bit** addition with carry in and carry out.
- It perform a single stage of this addition.

X	Y	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Full 1-bit Adder

► Example:

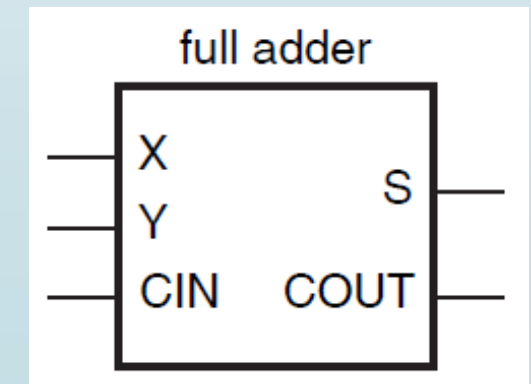
$$\begin{array}{r}
 A = 0001100 \\
 + B = 0011010 \\
 \hline
 0101010
 \end{array}$$

► Example:

► $A_2 = 1$ $B_2 = 1$ $C_{in} = 0$

► $S_2 = 0$ $C_{out} = 1$

X	Y	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Full 1-bit Adder

► Example:

$$\begin{array}{r}
 A = 0001 \quad 100 \\
 + B = 0011 \quad 110 \\
 \hline
 0101 \quad 010
 \end{array}$$

► Example:

► $A_2 = 1$

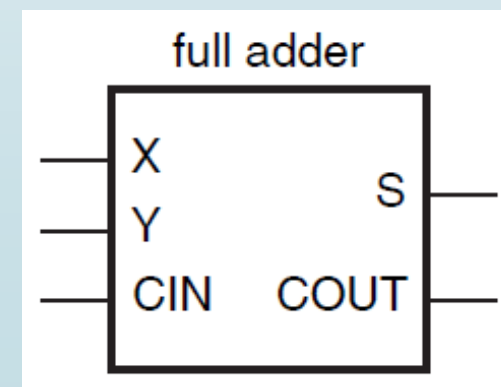
$B_2 = 0$

$C_{in} = 1$

► $S_2 = 0$

$C_{out} = 1$

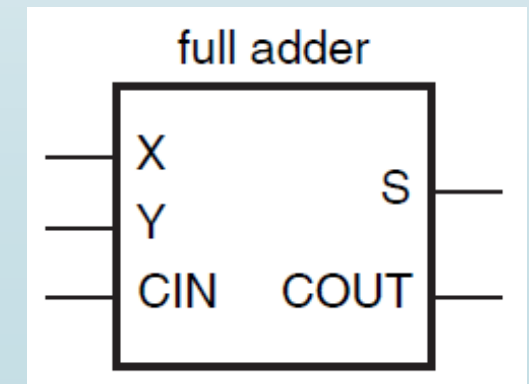
X	Y	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Full 1-bit Adder

- A full adder is used to add three bits, one of them is usually the carry in from previous stage, to form a sum bit and a carry out to the next stage.
- How to find the equation of Sum and carry out bits ??

X	Y	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



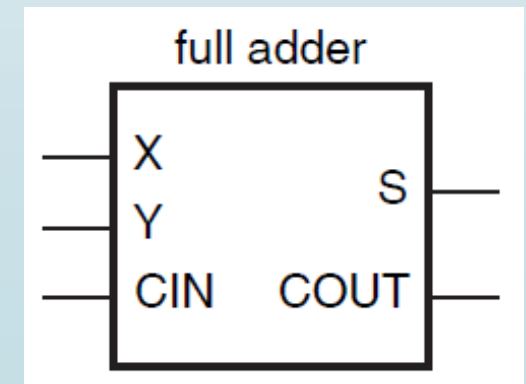
Full 1-bit Adder

- Plot K-map of Sum:

	X'	X	
Y'	0	1	C_{in}'
	1	0	C_{in}
Y	0	1	
	1	0	C_{in}'

- Equation: $S = ??$
- Diagonal ones, neglect K-map and find the XOR/XNOR form for minterms

X	Y	C_{in}	C_{out}
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



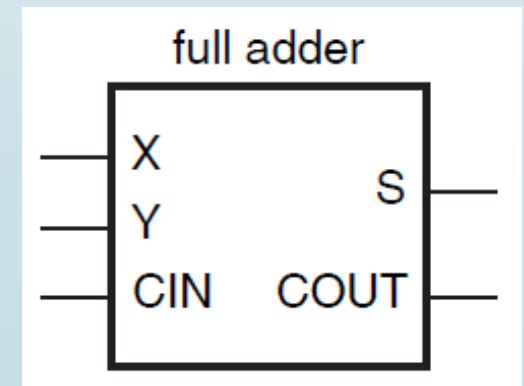
Full 1-bit Adder

► Active minterms of Sum:

$$S = X'Y'C_{in} + X'YC_{in}' + XY'C_{in}' + XYC_{in}$$

$$S = ??$$

X	Y	C_{in}	C_{out}
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



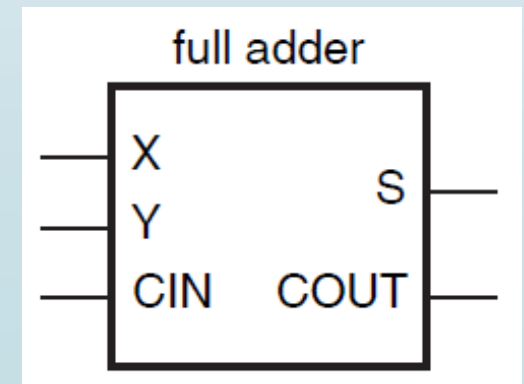
Full 1-bit Adder

Equation of Sum:

$$S = X'Y'C_{in} + X'YC_{in}' + XY'C_{in}' + XYC_{in}$$

$$S = X \oplus Y \oplus C_{in}$$

X	Y	C_{in}	C_{out}
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



Full 1-bit Adder

Equation of C_{out} :

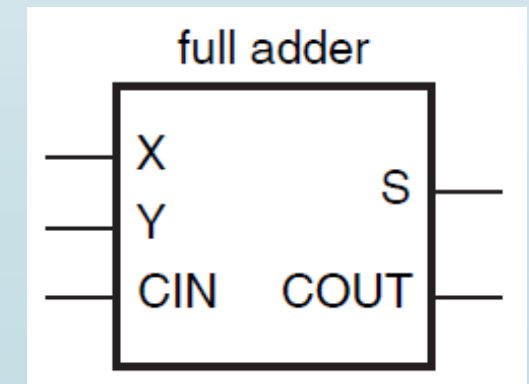
Solution (1): Use K-map:

	X'	X	
Y'	0	0	C_{in}'
	0	1	
	1	1	C_{in}
Y	0	1	C_{in}'

$$C_{out} = XY + XC_{in} + YC_{in}$$

$$C_{out} = XY + C_{in}(X+Y)$$

X	Y	C_{in}		Sum
0	0	0		0
0	0	1		1
0	1	0		1
0	1	1		0
1	0	0		1
1	0	1		0
1	1	0		0
1	1	1		1



Full 1-bit Adder

► Equation of C_{out} :

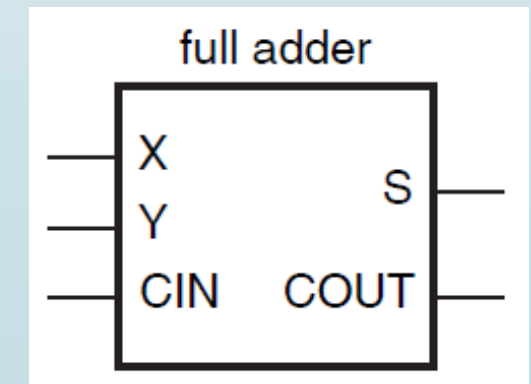
► Solution (2): Use active minterms:

$$C_{out} = X'YC_{in} + XY'C_{in} + XYC_{in}' + XYC_{in}$$

$$C_{out} = (X'Y + XY')C_{in} + XY(C_{in}' + C_{in})$$

$$C_{out} = C_{in}(X \oplus Y) + XY$$

X	Y	C_{in}		Sum
0	0	0		0
0	0	1		1
0	1	0		1
0	1	1		0
1	0	0		1
1	0	1		0
1	1	0		0
1	1	1		1



Full 1-bit Adder

- Equation of sum:

$$S = X \oplus Y \oplus C_{in}$$

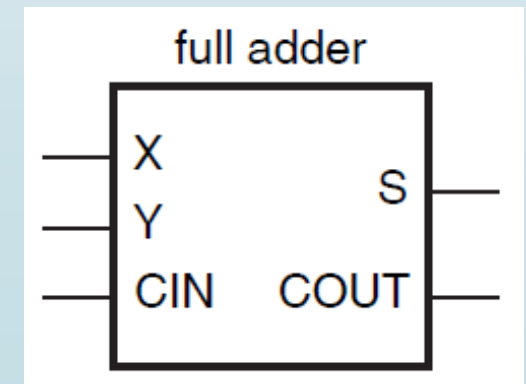
- Equations of carry out:

$$C_{out} = XY + XC_{in} + YC_{in}$$

$$C_{out} = XY + C_{in}(X+Y)$$

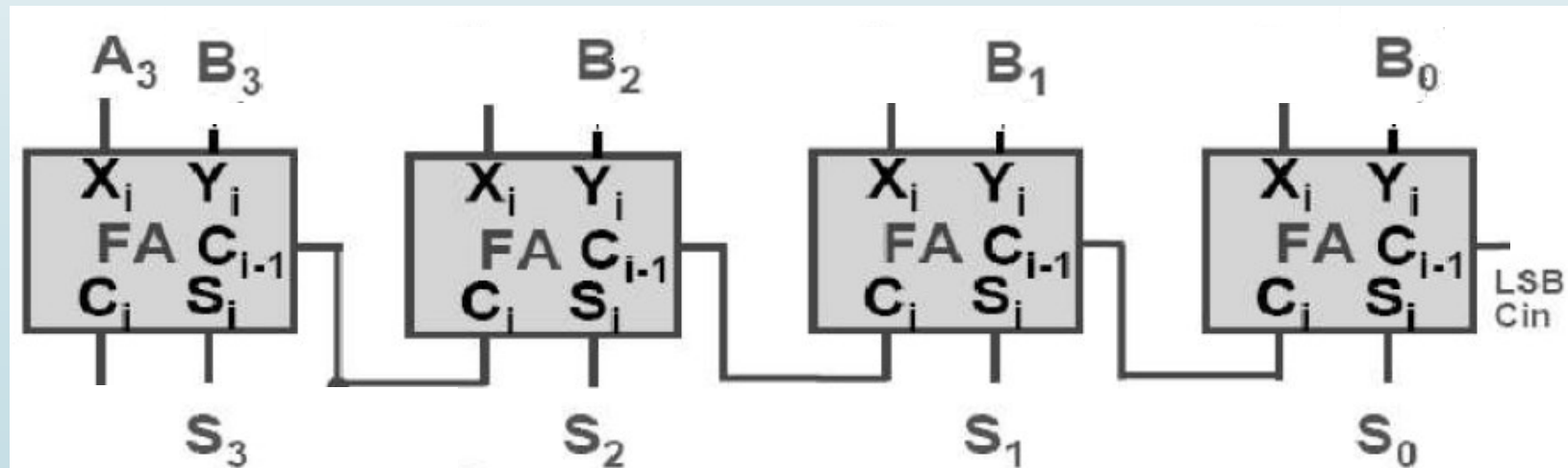
$$C_{out} = XY + C_{in}(X \oplus Y)$$

X	Y	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Ripple Adder

- The full 1-bit adder adds bits.
- Ripple added is used to add two words, each is n bits.
- It is a cascade of n full adders each of which handles 1 bit.
- The name "Ripple" describes the flow of carries.



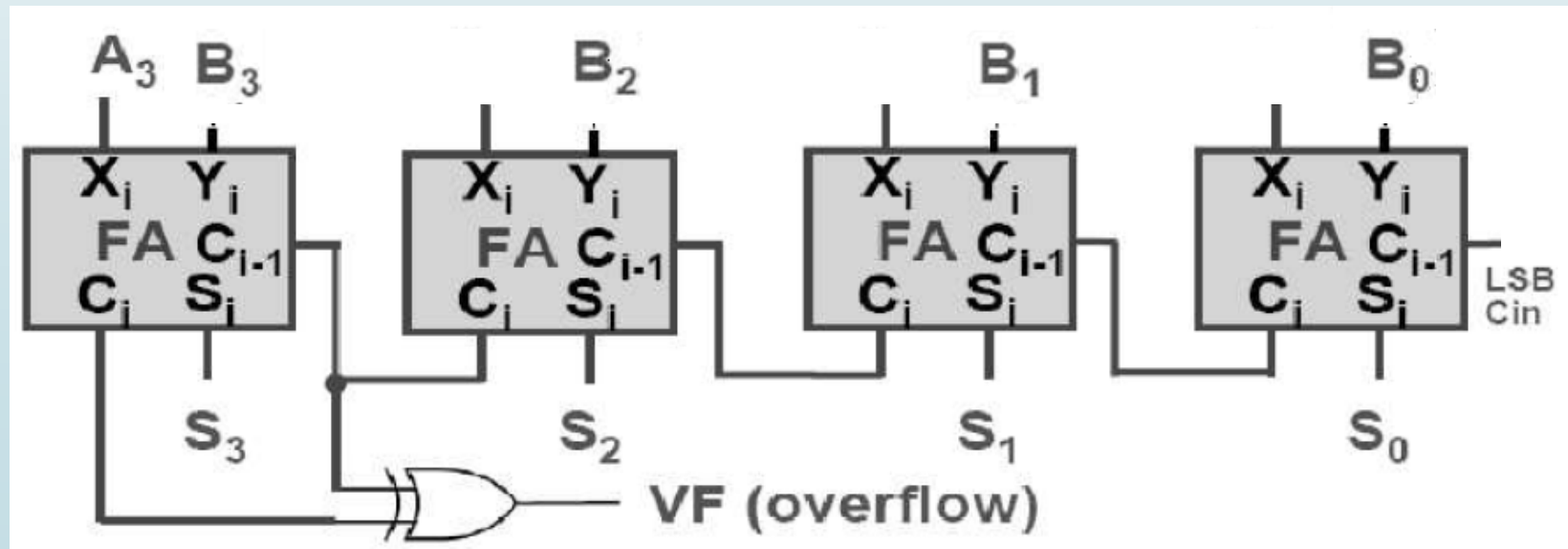
Flags

- Inside the CPU, there is the ALU (Arithmetic and Logic Unit).
- After the ALU perform any arithmetic logic operation, it updates some flags based on the result:
 - **Overflow flag** is a single bit = 1 if there is an overflow in the last operation, = 0 otherwise.
 - **Carry flag** is a single bit = 1 if there is a carry = 1 in the last operation, = 0 otherwise.
 - **Negative flag** is a single bit = 1 if the result of the last operation is negative, = 0 otherwise.
 - **Zero flag** is a single bit = 1 if the result of the last operation is zero, = 0 otherwise.

Ripple Adder/Subtractor

- Take the two's complement of the second number and add.

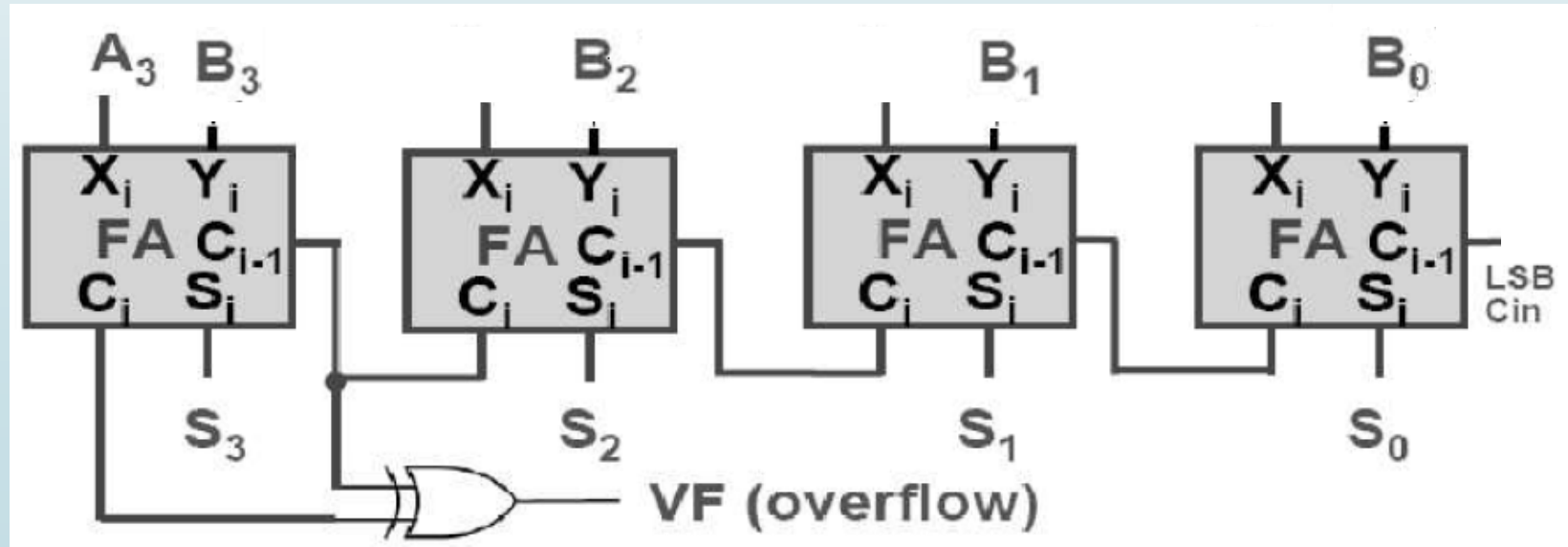
$$\begin{array}{r}
 +6 \quad 0 \ 1 \ 1 \ 0 \\
 - (+7) \quad 0 \ 1 \ 1 \ 1 \\
 \hline
 \end{array}
 \longrightarrow
 \begin{array}{r}
 \quad \quad \quad 0 \ 1 \ 1 \ 0 \\
 \quad \quad \quad 1 \ 0 \ 0 \ 0 \\
 \quad \quad \quad \quad \quad 1 \\
 \hline
 = -1 \quad 1 \ 1 \ 1 \ 1
 \end{array}$$



Ripple Adder/Subtractor

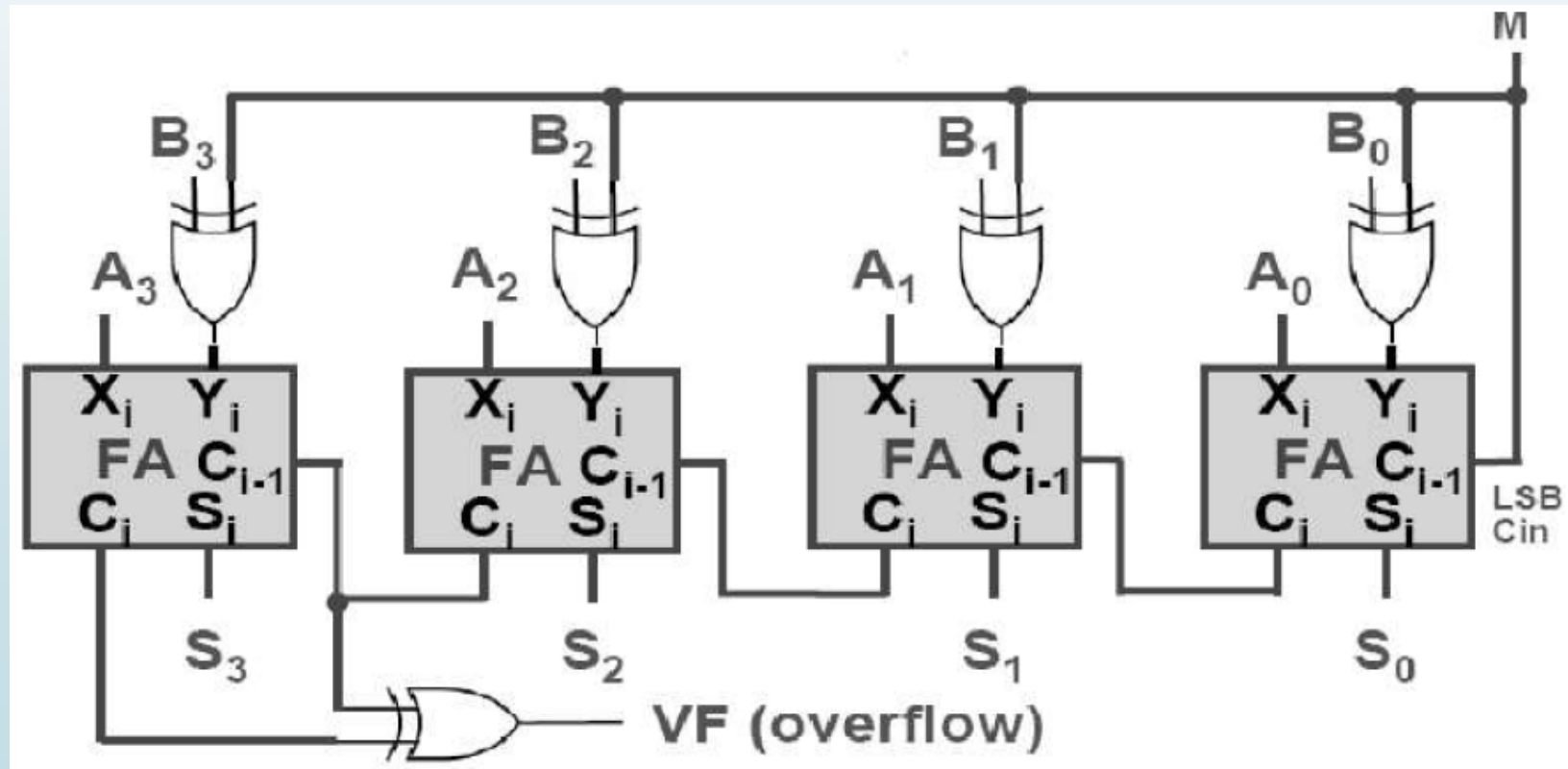
- In other words, flip each bit in the second number and add 1

$$\begin{array}{r}
 +6 \quad 0 \ 1 \ 1 \ 0 \\
 - (+7) \quad 0 \ 1 \ 1 \ 1 \\
 \hline
 \end{array}
 \longrightarrow
 \begin{array}{r}
 \quad 0 \ 1 \ 1 \ 0 \\
 \quad 1 \ 0 \ 0 \ 0 \\
 1 \\
 \hline
 = -1 \quad 1 \ 1 \ 1 \ 1
 \end{array}$$



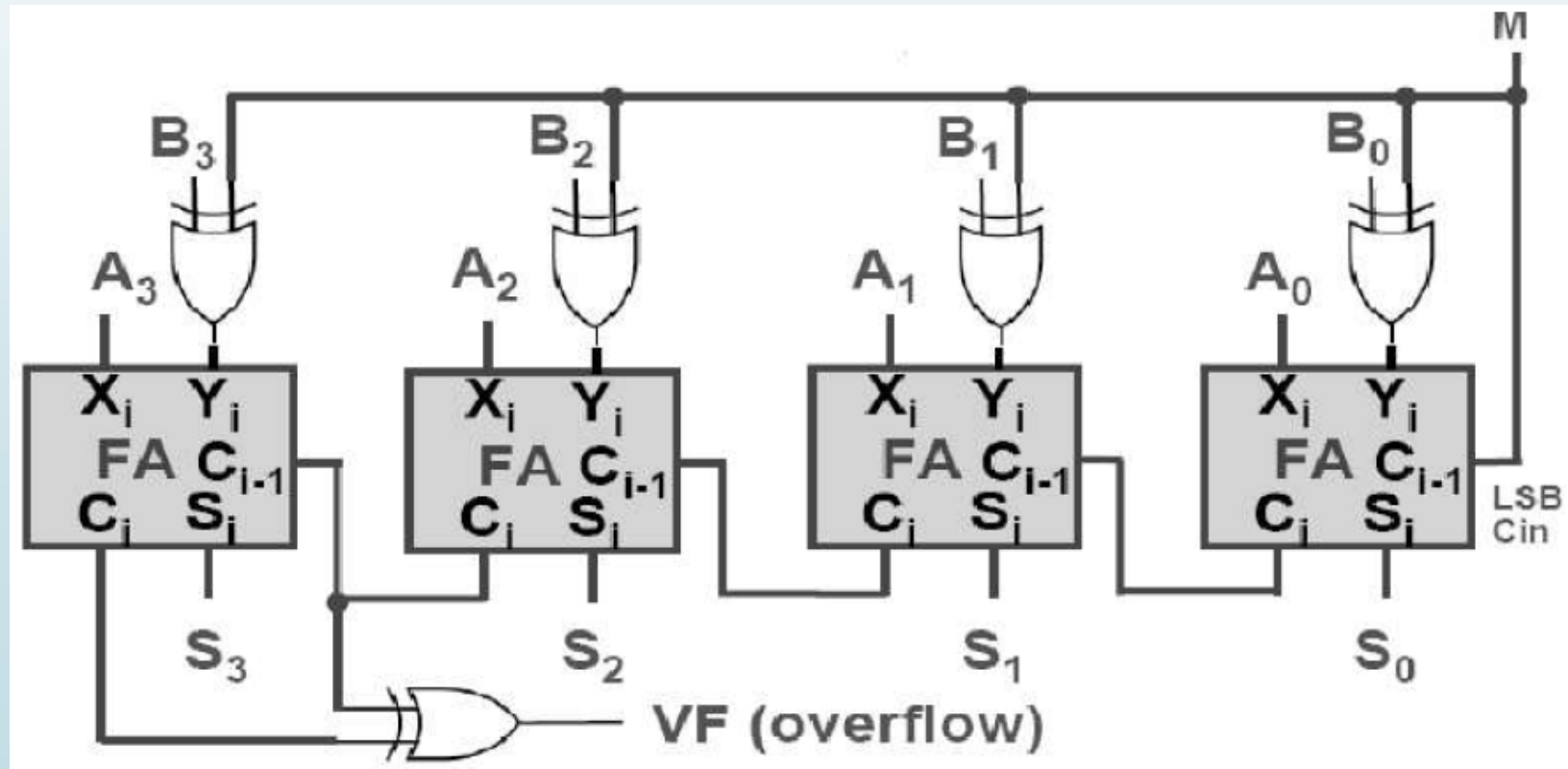
Ripple Adder/Subtractor

- Remember: $X \oplus 0 = X$ $X \oplus 1 = X'$
- To subtract, put $M = 1$: The XOR is inverted and $C_{in} = 1$



Ripple Adder/Subtractor

- Remember: $X \oplus 0 = X$ $X \oplus 1 = X'$
- To add, put $M = 0$: The XOR is buffer and no C_{in}



Carry Flag

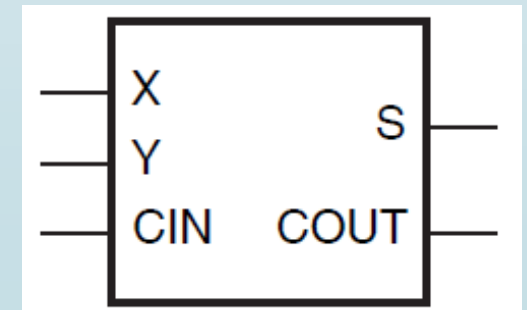
- To detect if there is carry happened, we can check the last C_{out} of the last full adder.
 - If $C_{out} = 0$, there is no carry from the whole addition.
 - If $C_{out} = 1$, there is a carry from the whole addition.

Half Adder

Remember:

- A full adder is used to add three bits, one of them is usually the carry in from previous stage, to form a sum bit and a carry out to the next stage.

X	Y	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Half Adder

- A half adder is used to add two bits, to form a sum bit and a carry.

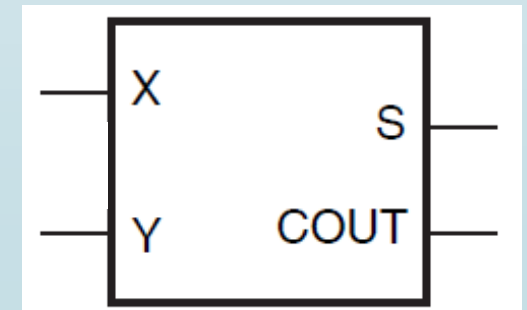
- Equations:

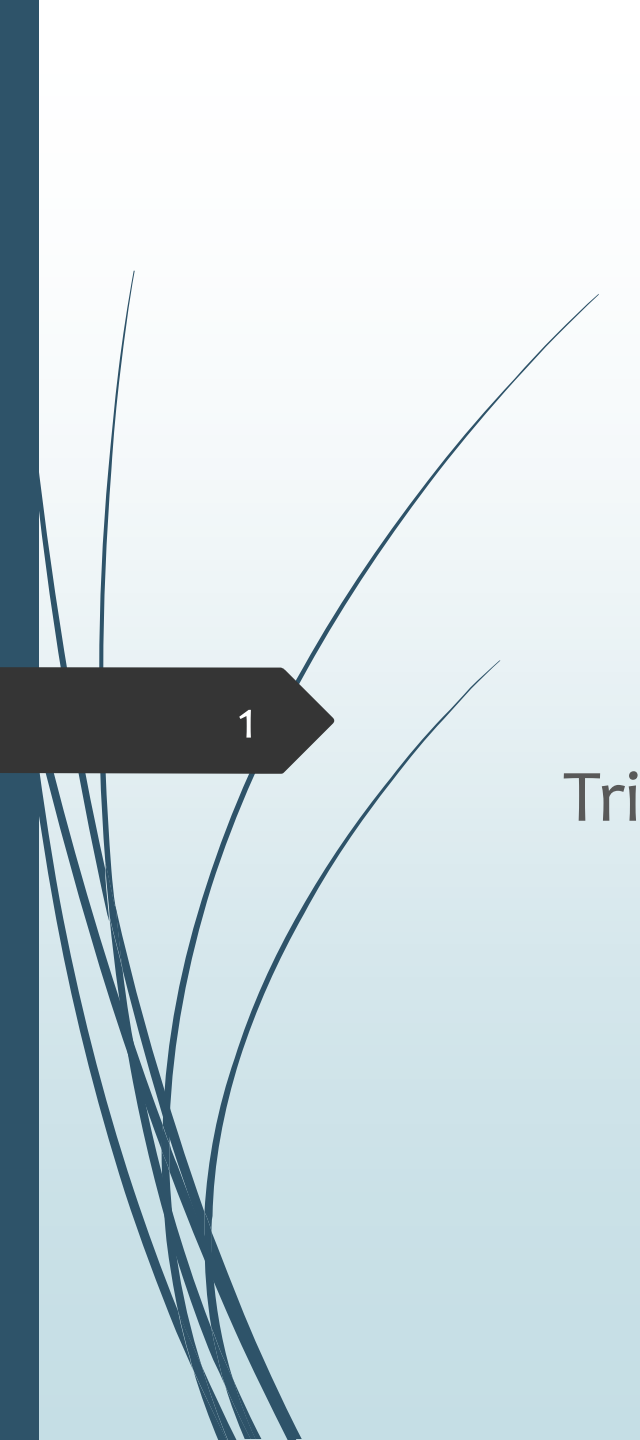
$$S = X \oplus Y$$

$$C_{out} = XY$$

- It can be used in any application when we need to implement the same equations.

<i>X</i>	<i>Y</i>	<i>C_{out}</i>	<i>Sum</i>
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

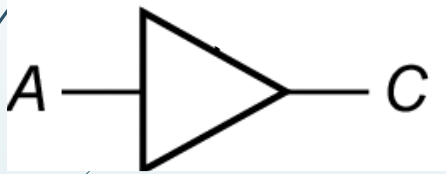




1

Tri-state (three-state) Buffer

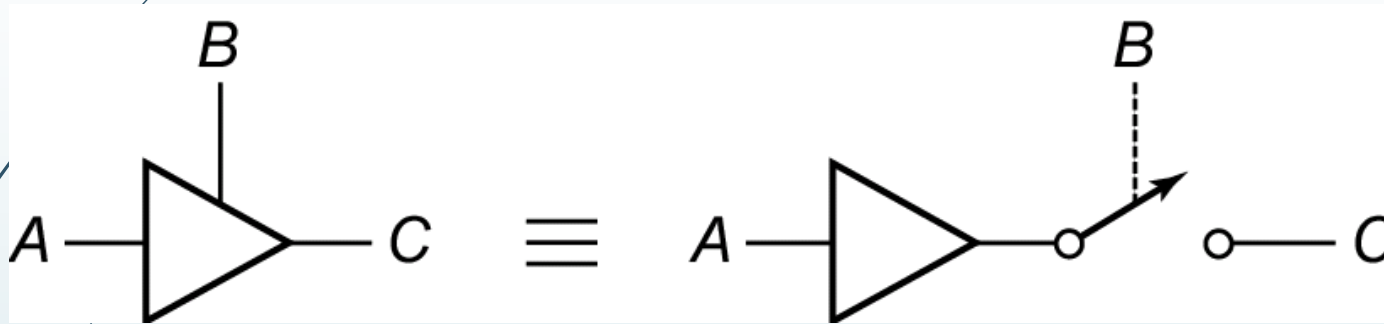
Buffer



► The buffer equation: $C=A$.

A	C
0	0
1	1

Tri-state Buffer



B	C
0	Hi-Z
1	A

- When the enable input B of a three-state buffer is 1 the output $C=A$.
- When the enable input is 0, the output C acts like an open circuit, commonly known as *high impedance*.
- The three states of the buffer output are: 1, 0, and Hi-Z.

A	B	C
0	0	Hi-Z
0	1	0
1	0	Hi-Z
1	1	1

Application: Party Lines

- Assume that we are sending data from several sources (PQRSTU VW) to a single destination (SDATA) and vice versa.

P—

Q—

R—

—— SDATA

S—

T—

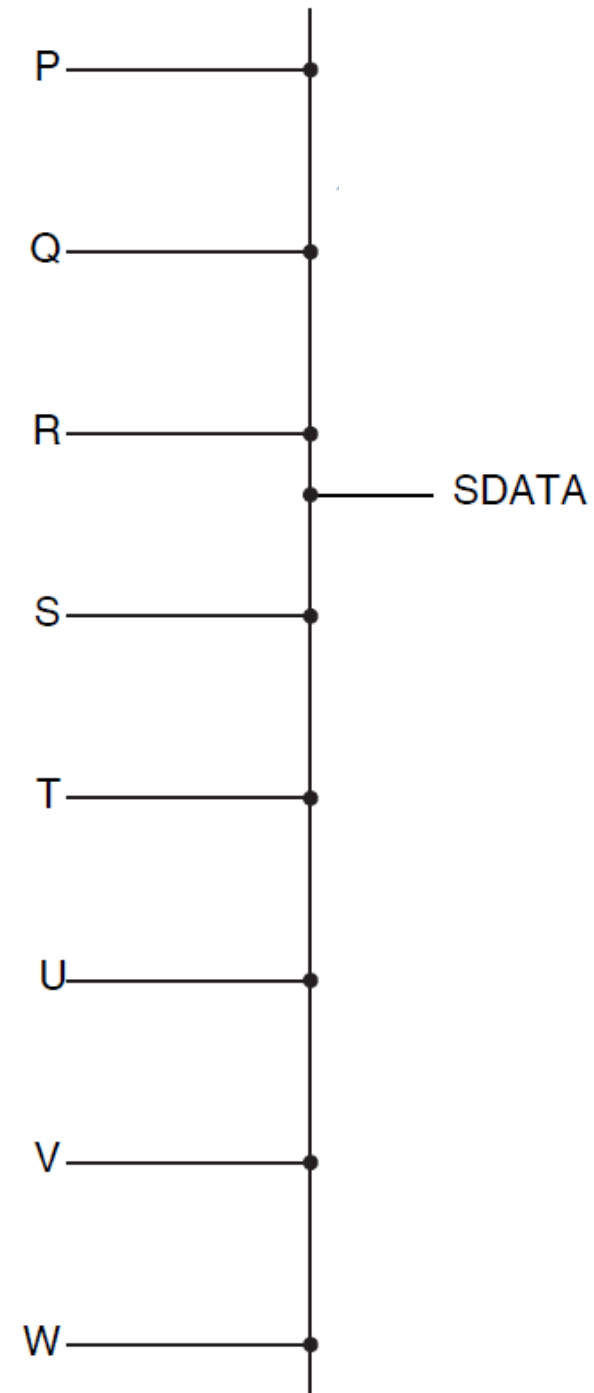
U—

V—

W—

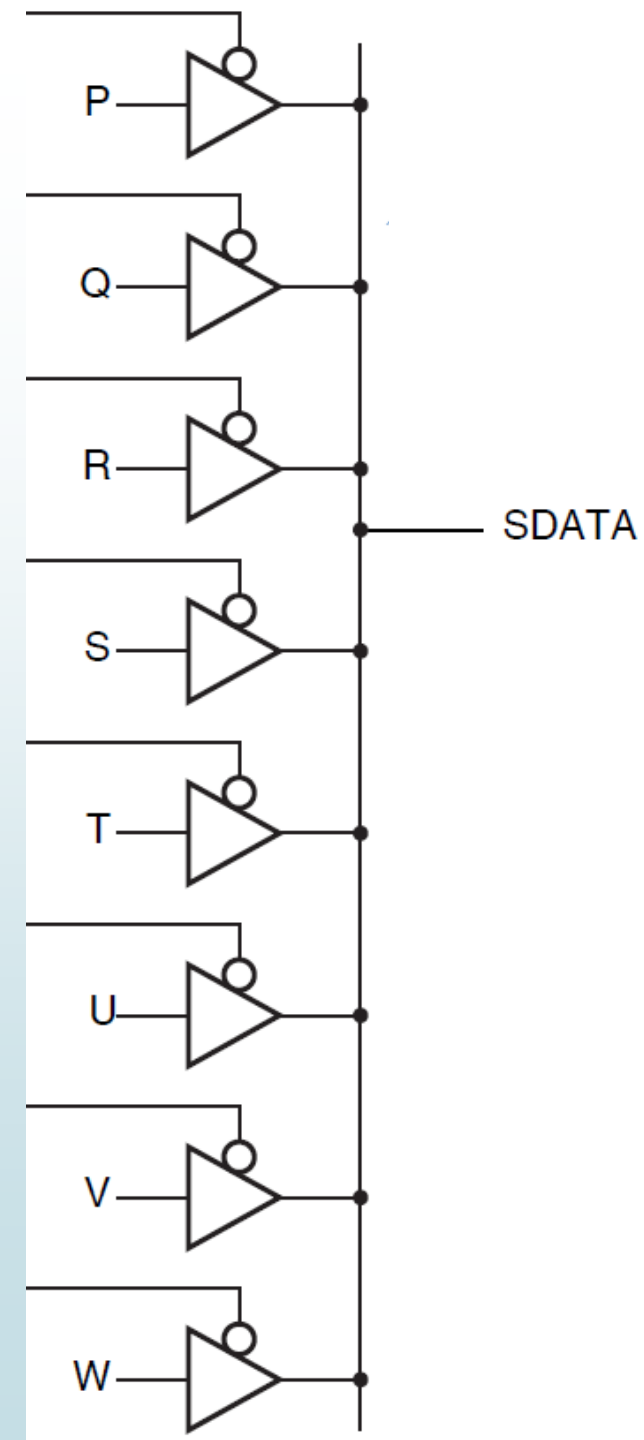
Application: Party Lines

- Assume that we are sending data from several sources (PQRSTU VW) to a single destination (SDATA).
- Party line is a single line that all the sources use for communication.
- This allows only 1-bit transfer at a time (serial data).
- However, we need to organize the access to the line so that no two sources will send data simultaneously.



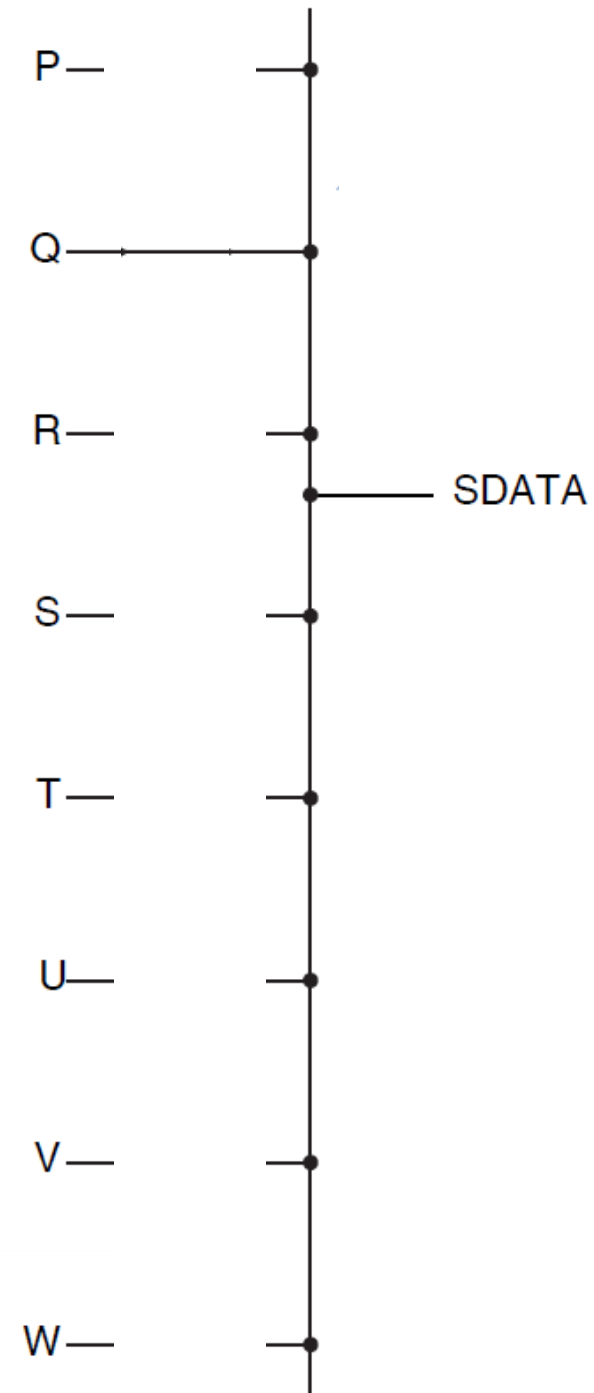
Application: Party Lines

- This is done using tri-state buffers.
- To allow the source Q to send, we put 0 on its tri-state buffer and 1 on the others (active low).



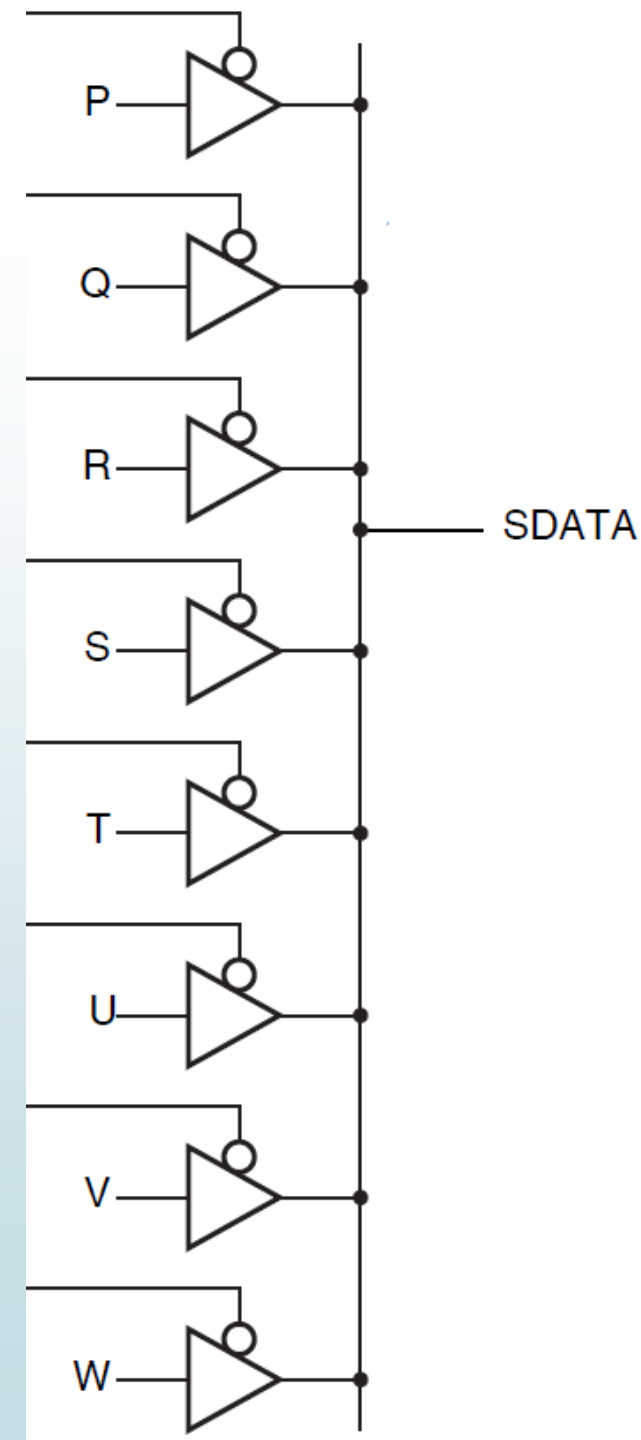
Application: Party Lines

- This is done using tri-state buffers.
- To allow the source Q to send, we put 0 on its tri-state buffer and 1 on the others (active low).



Application: Party Lines

- This is done using tri-state buffers.
- To allow the source Q to send, we put 0 on its tri-state buffer and 1 on the others (active low).
- Generally, to allow a single source to send, we put 0 on its tri-state buffer and 1 on the others.
- How, do you think, could we implement this case ??



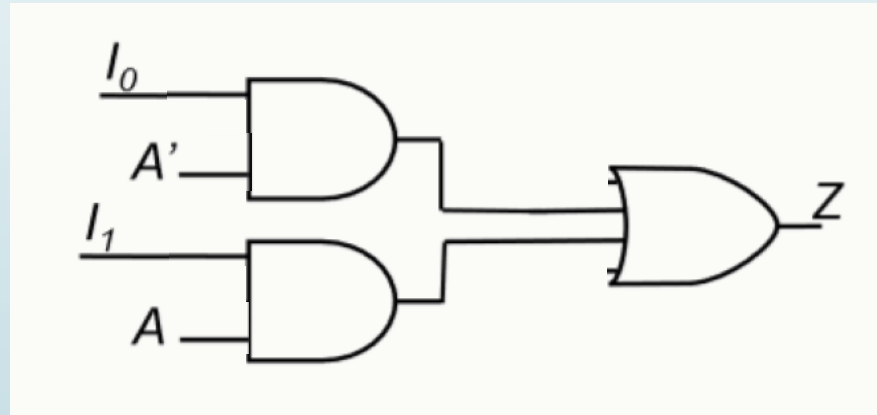


10

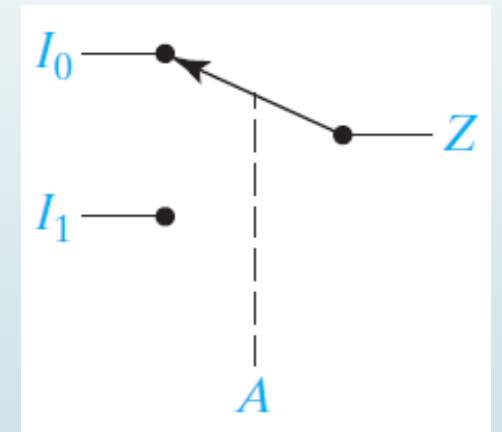
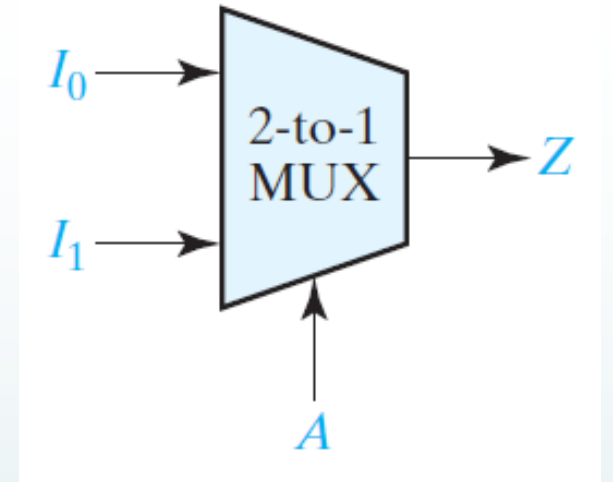
Multiplexer (MUX)

2-to-1 Multiplexer

- If $A=0$, then $Z=I_0$
- If $A=1$, then $Z=I_1$
- $Z=A' I_0 + A I_1$
- Circuit:



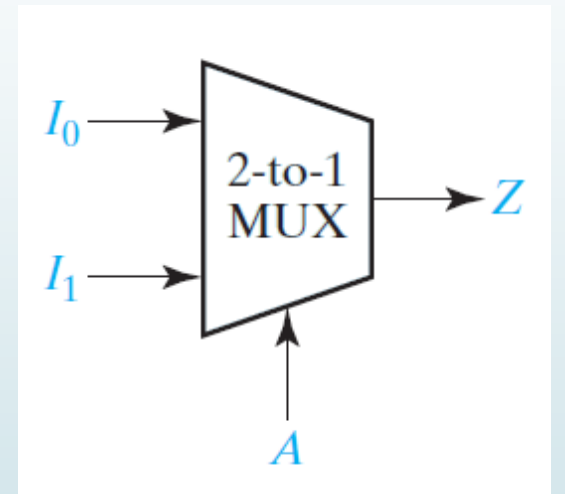
- The control lines are used to select one of the inputs and connect it to the output.



A	Z
0	I_0
1	I_1

2-to-1 Multiplexer

- Tri-state buffers can be used to implement the MUX.
 - If $A=0$, then $Z=I_0$
 - If $A=1$, then $Z=I_1$

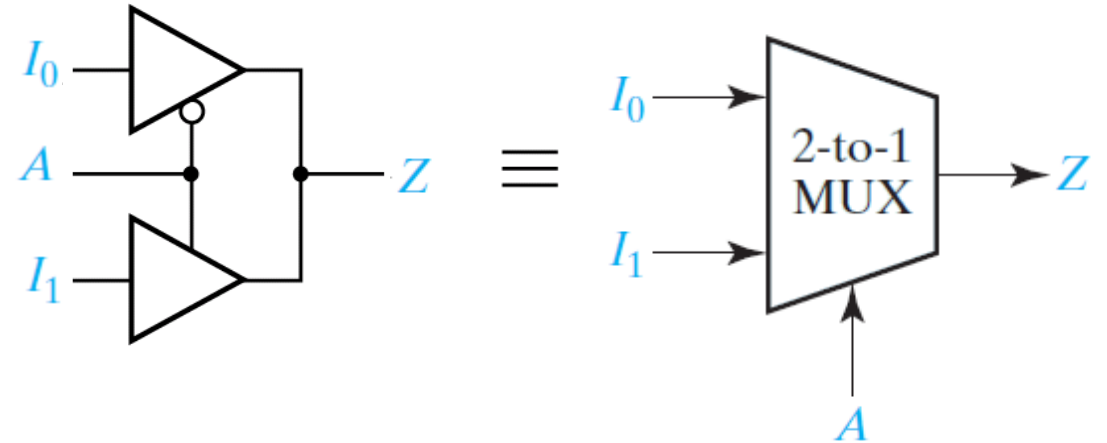


2-to-1 Multiplexer

► Tri-state buffers can be used to implement the MUX.

► If $A=0$, then $Z=I_0$

► If $A=1$, then $Z=I_1$



► Put a tri-state buffer on each input (gate):

► When $A=0$: the top buffer is enabled $\rightarrow Z=I_0$

► When $A=1$: the bottom buffer is enabled $\rightarrow Z=I_1$

4-to-1 Multiplexer

➤ Similarly:

➤ Symbol:

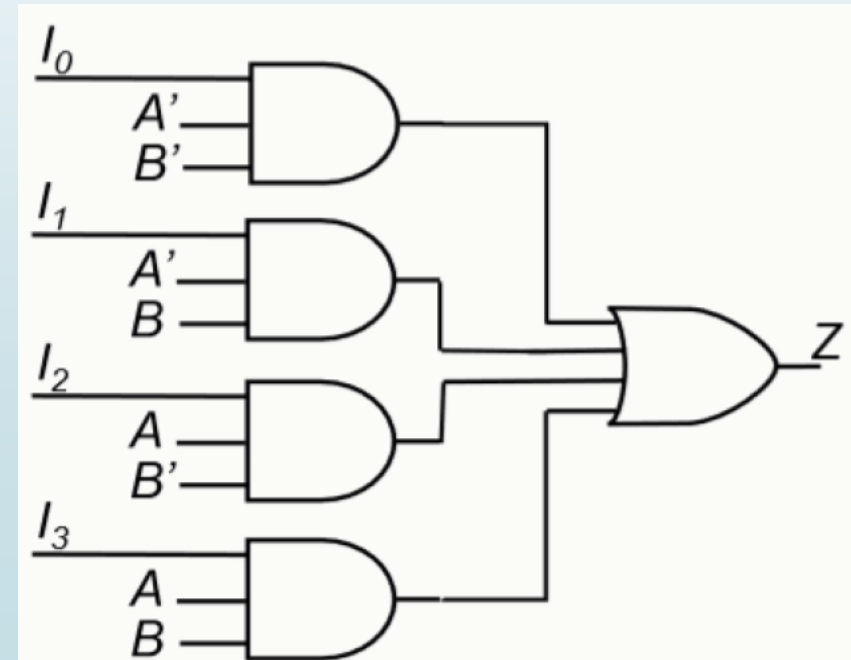
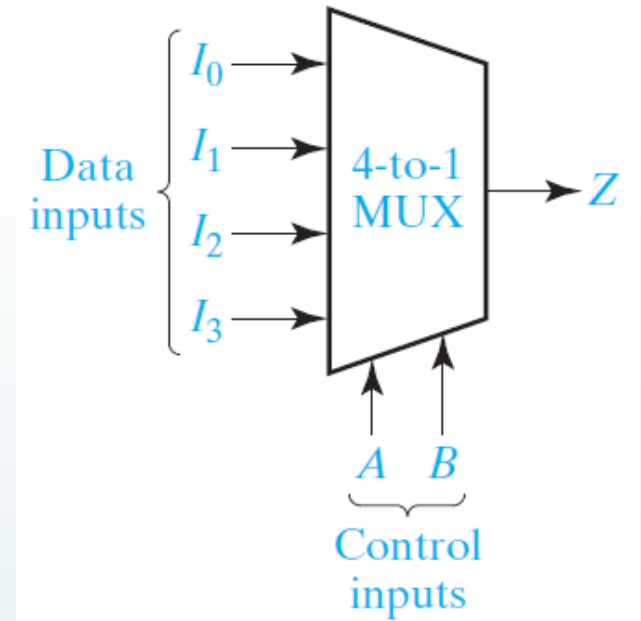
➤ Truth table:

A	B	Z
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

➤ Equation:

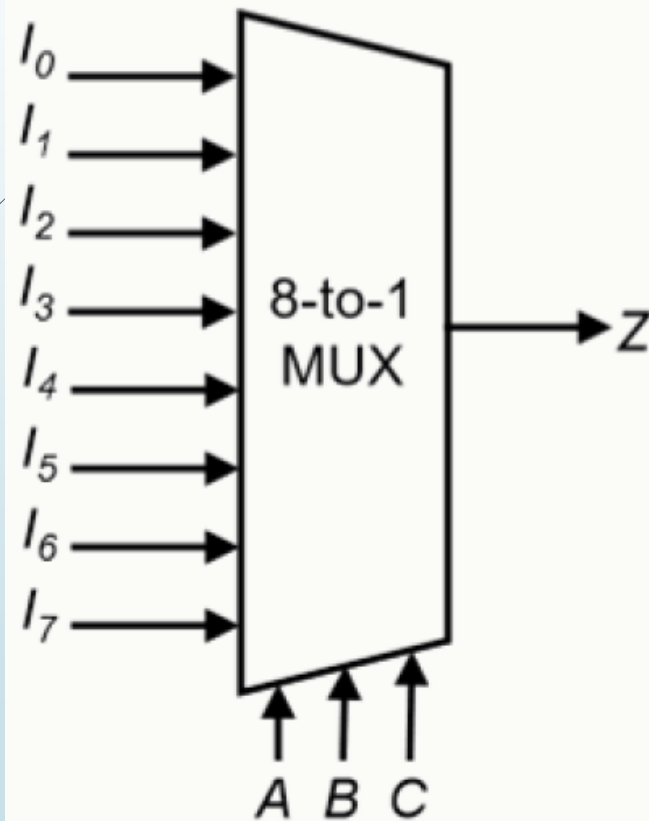
$$Z = A'B'I_0 + A'BI_1 + AB'I_2 + ABI_3$$

➤ Circuit:

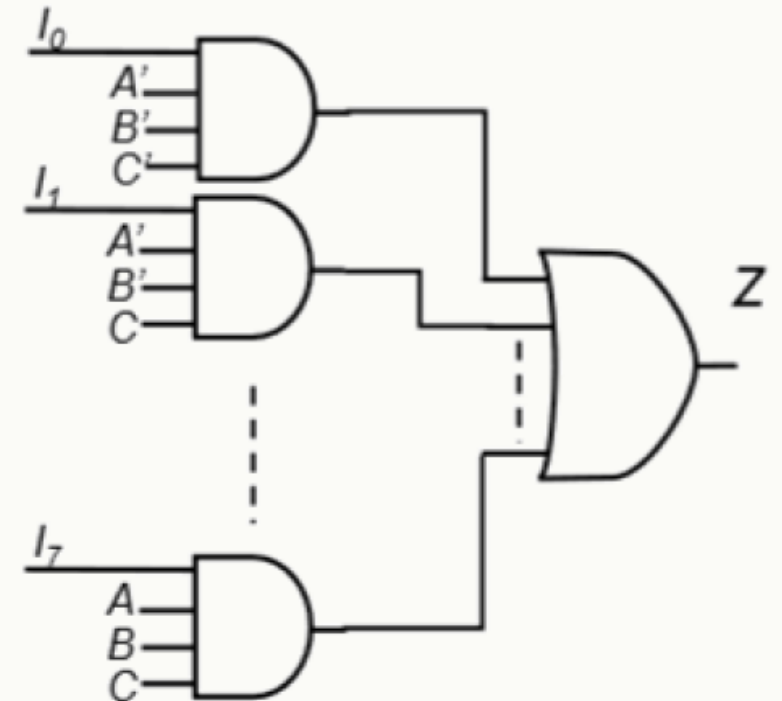


8-to-1 Multiplexer

Similarly:



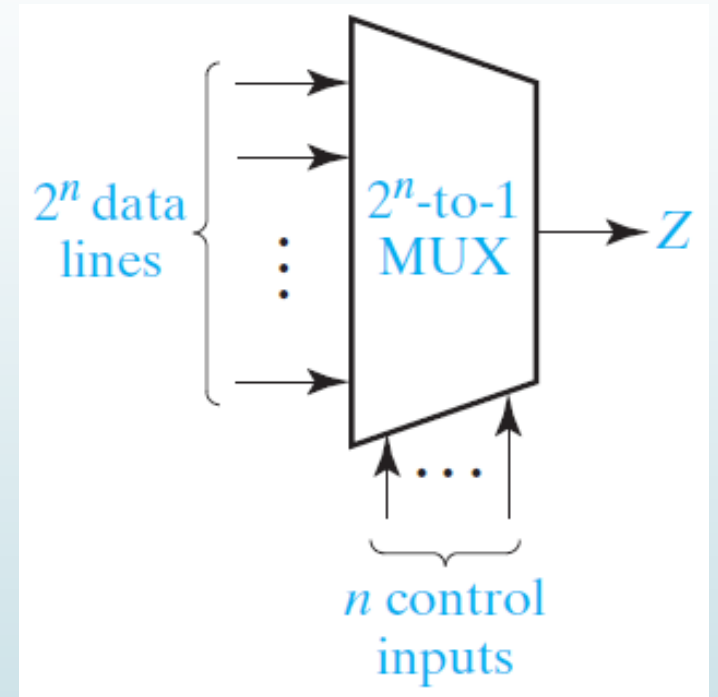
A	B	C	Z
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7



$$Z = A'B'C'I_0 + A'B'CI_1 + AB'C'I_2 + A'BCI_3 + AB'C'I_4 + AB'CI_5 + ABC'I_6 + ABCI_7$$

Generally

- A 2^n -to-1 multiplexer has:
 - 2^n inputs
 - n control lines
 - One output
- $n=1$: 2-to-1 MUX
- $n=2$: 4-to-1 MUX
- $n=3$: 8-to-1 MUX
- ...



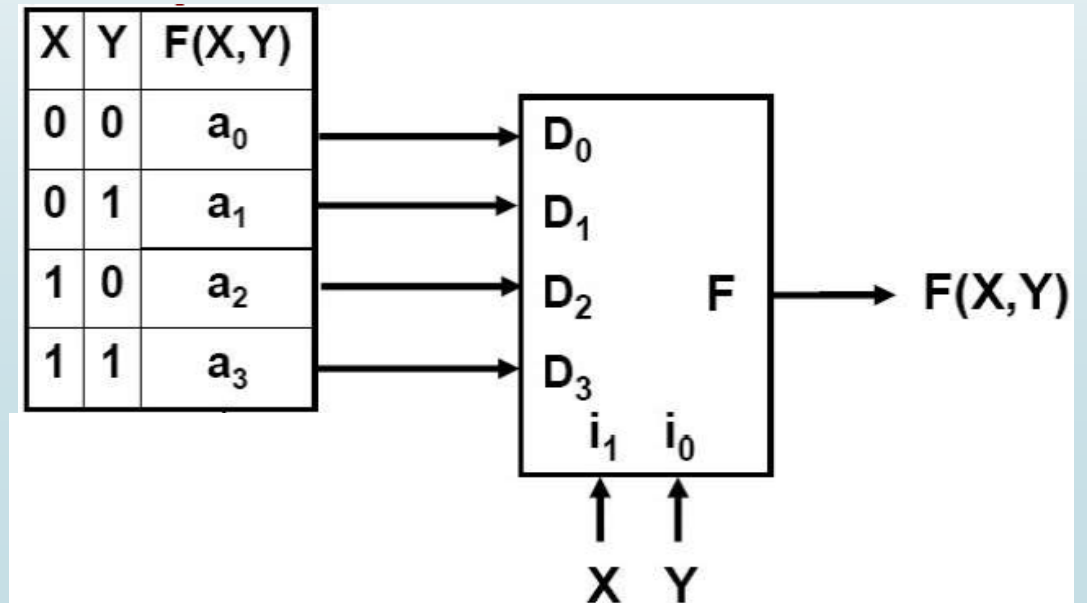
Truth Table Analogy

- Application: Multiplexers can be used as another way to realize logic functions.
- Given the truth table of an n-variable logic function ($n=2$).

X	Y	F(X,Y)
0	0	a_0
0	1	a_1
1	0	a_2
1	1	a_3

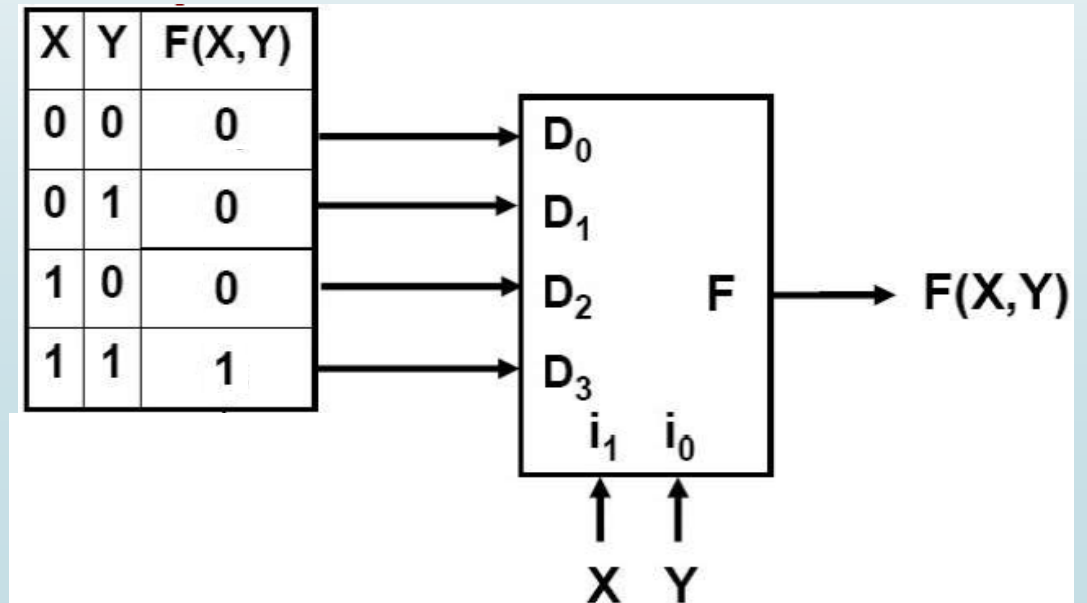
Truth Table Analogy

- Put the table output values on the inputs of 2^n -to-1 MUX.
- Apply the function input variables (X, Y) as control (i_0, i_1).



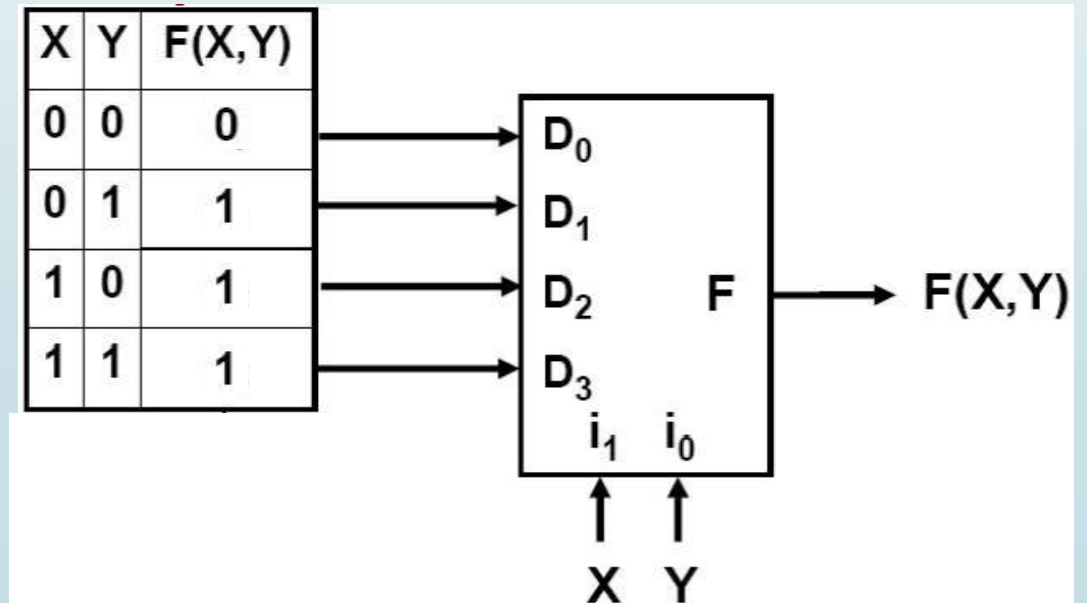
Truth Table Analogy

- Put the table output values on the inputs of 2^n -to-1 MUX.
- Apply the function input variables (X, Y) as control (i_0, i_1).
- Example: To implement X AND Y gate:
 - Connect X, Y lines to MUX controls.
 - Apply $a_0=0$ on D_0
 - Apply $a_1=0$ on D_1
 - Apply $a_2=0$ on D_2
 - Apply $a_3=1$ on D_3



Truth Table Analogy

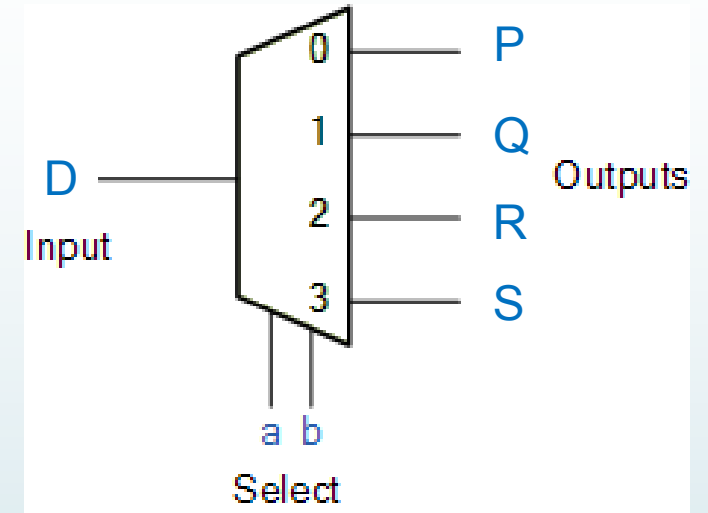
- Put the table output values on the inputs of 2^n -to-1 MUX.
- Apply the function input variables (X, Y) as control (i_0, i_1).
- Example: To implement X OR Y gate:
 - Connect X, Y lines to MUX controls.
 - Apply $a_0=0$ on D_0
 - Apply $a_1=1$ on D_1
 - Apply $a_2=1$ on D_2
 - Apply $a_3=1$ on D_3



Demultiplexer (DMUX)

Demultiplexer

- The function of a de-multiplexer is just the inverse of a multiplexer.
- Based on the control lines (a,b), route the input (D) to one of the outputs (P,Q,R,S).



Select		Outputs			
a	b	P	Q	R	S
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

Application: Driving a Bus

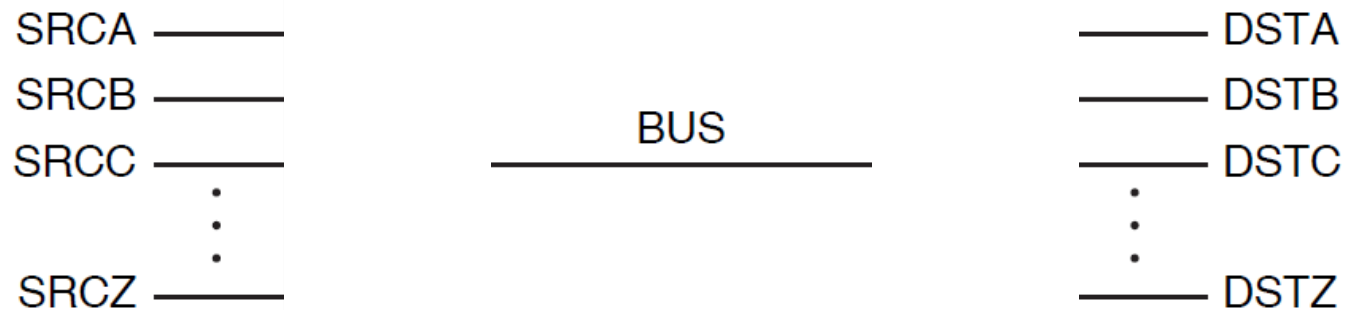
- Assume that we have many sources SRCA-SRCZ.
- Each source can send one of many destinations DSTA-DSTZ

SRCA —
SRCB —
SRCC —
⋮
SRCZ —

— DSTA
— DSTB
— DSTC
⋮
— DSTZ

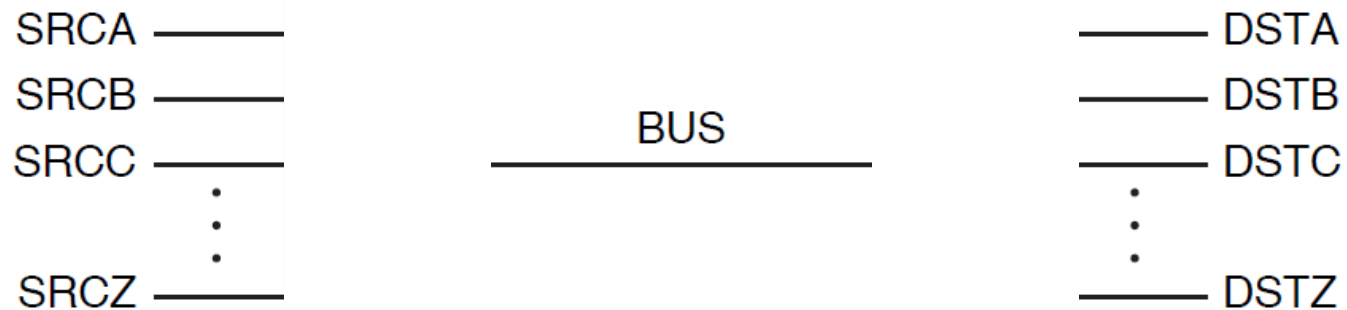
Application: Driving a Bus

- Assume that we have many sources SRCA-SRCZ.
- Each source can send one of many destinations DSTA-DSTZ
- Assume that only one bus is available for all the sources and distinctions.



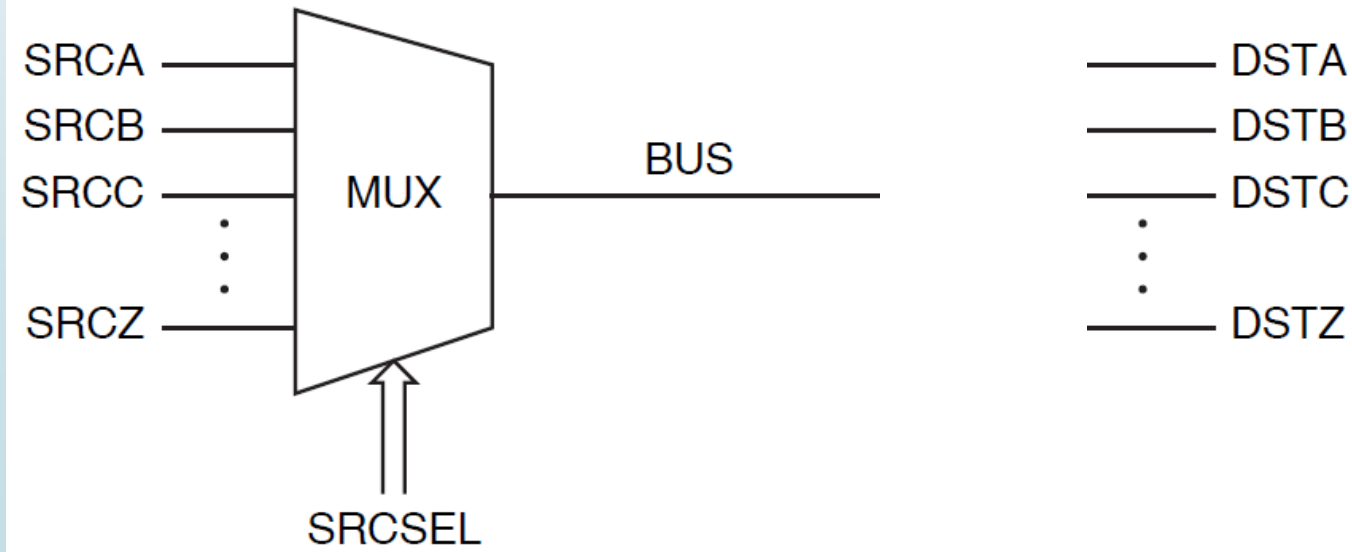
Application: Driving a Bus

- We need to make sure that only one source is sending and only one destination is receiving at a time.
- How do you think could we implement this??



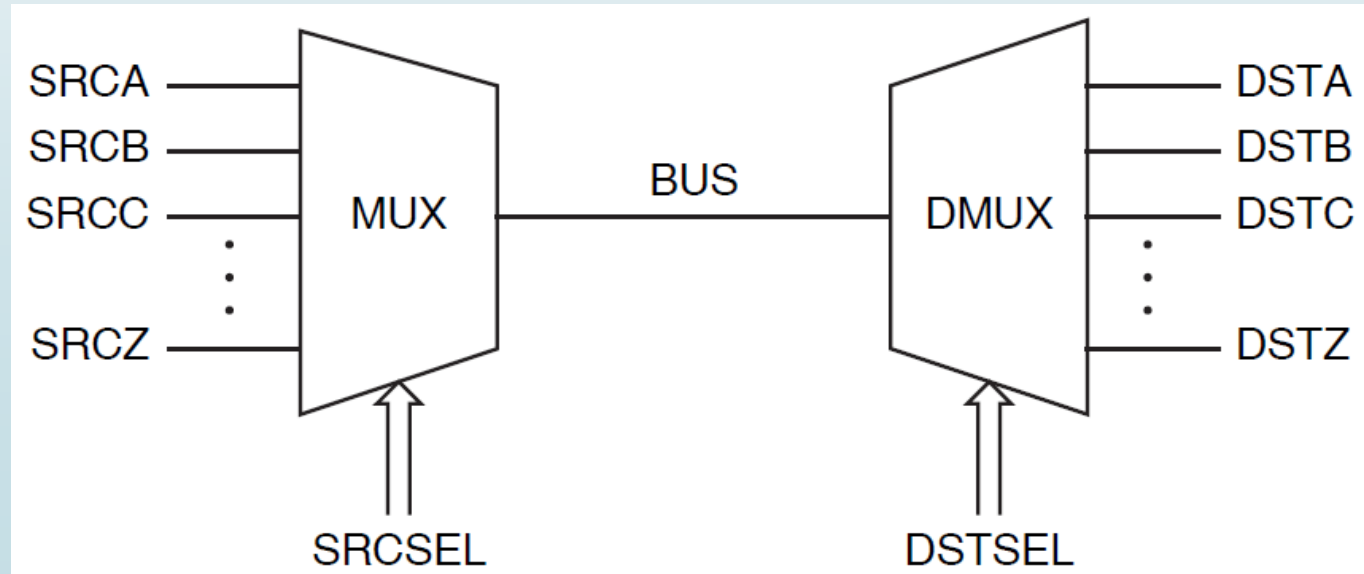
Application: Driving a Bus

- Implement a MUX on the sources.
- Based on the MUX control lines we can select which source to send now.
- Example: If $\text{SRCSEL} = 00010$, then contents of SRCC are put on BUS.



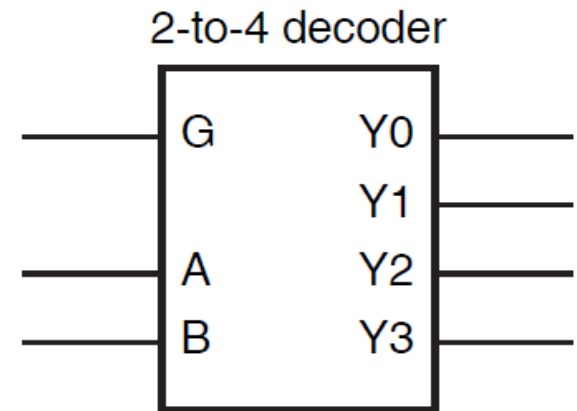
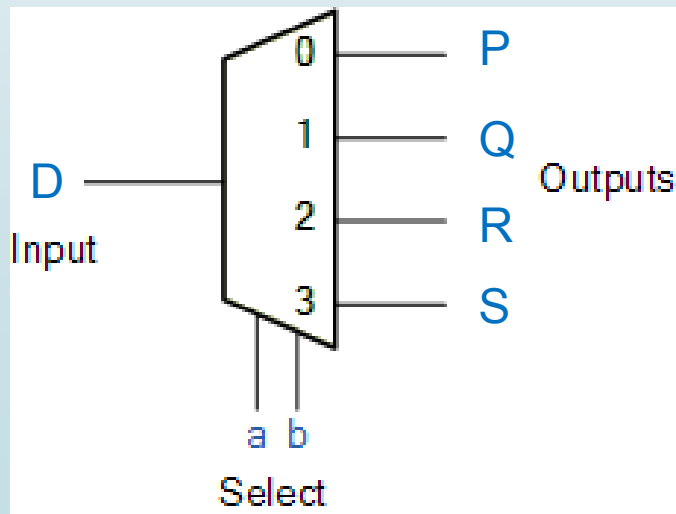
Application: Driving a Bus

- Implement a DMUX on the destinations.
- Based on the DMUX control lines we can select which destination to receive the sent data.
- Example: If $DSTSEL = 00001$, then $DSTB = SRCC$ over the BUS.



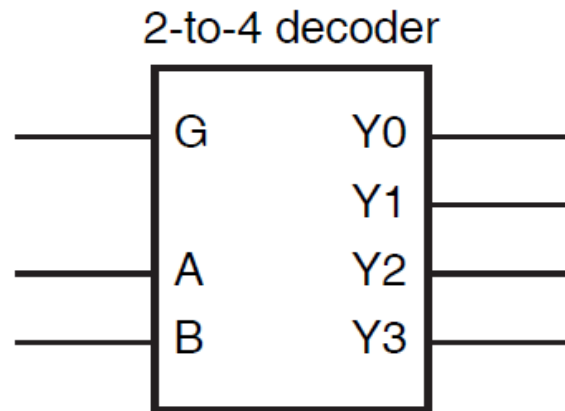
Demultiplexer

- A binary decoder with an enable input can be used as a demultiplexer.
- To implement a 1-to-n DMUX we need a 2-to- 2^n decoder.
 - Example: To implement a 1-to-4 DMUX we need a 2-to-4 decoder with enable G.



Demultiplexer

- Consider the truth table of a 2-to-4 decoder with active high enable G.
- When $G=0$, the decoder outputs are all zeros.



A	B	G	Y0	Y1	Y2	Y3
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	0	0
0	1	1	0	1	0	0
1	0	0	0	0	0	0
1	0	1	0	0	1	0
1	1	0	0	0	0	0
1	1	1	0	0	0	1